

ia-bdd-agent

Do chamado Bitrix24 ao cenário BDD em Gherkin

Victor Martins da Silva · QA Mobilemed

Node.js · Bitrix24 REST · Gherkin (pt-BR)

O problema

Hoje, sem o agente

- Card entra em **Novo Teste** → QA monta cenários **manualmente**
- Informação espalhada: **Descrição, Passos, Cenários Dev**
- Formato **inconsistente** entre squads (DICOM, Sustentação, Portal...)
- Risco de **perder tempo** ou **sobrescrever** cenários já aprovados

A solução

O que o ia-bdd-agent faz

1. **Monitora** a fila QA no Bitrix24
2. **Lê** descrição, passos, resultados e **Cenários Dev**
3. **Gera** BDD em português (Gherkin: Dado / Quando / Então)
4. **Valida e repara** estrutura inválida (sem `E cenário:`, Então incompleto)
5. **Grava** no campo **Cenários QA** do CRM
6. **Salva** `.feature` em `output/` para revisão

Onde atua

Item	Configuração
SPAs	1276 (NGF) + 1294 (ex.: DICOM)
Esteiras	DICOM, Sustentação, Improve, Desenvolvimento QA...
Colunas	Novo Teste, Teste de Q.A., Pronto para teste
Destino CRM	ufCrm100CenariosQa / ufCrm94CenariosQa
Não grava em	Campo Dev nem coluna de desenvolvimento

Fluxo do sistema

```
Bitrix24 (fila QA)
  ↓
poll.js (~30s)  ou  bdd:item <id>
  ↓
Lê campos do card (NGF + Dev + evidências)
  ↓
Gerador estruturado + refino OpenAI (opcional)
  ↓
Reparo + rigor + planner
  ↓
Campo vazio ou inválido? → grava CRM + .feature
Campo aprovado?           → ignora (protege QA)
```

Arquitetura (camadas)

Camada	Arquivos principais
Entrada	<code>poll.js</code> , <code>index.js</code> , <code>scripts/run-bdd-item.js</code>
Orquestração	<code>run-bitrix-bdd-cycle.js</code>
Agentes	<code>bdd.agent.js</code> , <code>parser.js</code>
Bitrix/CRM	<code>bitrix.service.js</code> , <code>push-bdd-to-crm.js</code>
Qualidade Gherkin	<code>bdd-gherkin.js</code> , <code>bdd-gherkin-structure.js</code> , <code>bdd-rigor.js</code>
IA	<code>ia.service.js</code> , <code>prompts/*.txt</code>

Geração de BDD — pipeline

1. Estruturado (padrão, fiel ao CRM)
Dev → QA · NGF · parser de título



2. Refino OpenAI (opcional)
bdd-refine.txt



3. Pós-processamento
reparar → rigorizar → CRM limpo

Fontes do cenário (prioridade)

1. **Cenários de Teste (Dev)** — cada bloco vira cenário QA
2. **Passos para reproduzir** — Quando / E
3. **Resultado esperado / obtido** — Então assertivo
4. **Evidências Dev** — análise visual (OpenAI Vision, opcional)
5. **Título do card** — parser inteligente (ex.: CNPJ + Pessoa Física)

Modo **LLM** opcional; padrão = **estruturado** (mais previsível).

Arquivos-chave — agentes e serviços

Arquivo	Função
<code>src/agents/bdd.agent.js</code>	Orquestra geração, refino LLM, validação
<code>src/agents/parser.js</code>	Extraí contexto do item CRM
<code>src/services/bitrix.service.js</code>	API Bitrix (list, get, update)
<code>src/services/push-bdd-to-crm.js</code>	Grava Cenários QA ; classifica generate/skip
<code>src/services/ia.service.js</code>	OpenAI / Ollama
<code>src/orchestrator/run-bitrix-bdd-cycle.js</code>	Ciclo completo por card

Arquivos-chave — qualidade Gherkin

Arquivo	Função
<code>bdd-gherkin.js</code>	Monta passos, cenários Dev→QA, parser de título
<code>bdd-gherkin-structure.js</code>	Valida e repara Gherkin desconexo
<code>bdd-rigor.js</code>	Remove passos vagos; Então verificável
<code>bdd-scenario-planner.js</code>	Ordena, deduplica, lacunas
<code>bdd-crm-merge.js</code>	BDD limpo no CRM (sem <code>#</code> , sem append IA)
<code>bdd-validacoes.js</code>	Critérios do resultado esperado
<code>bdd-prompts.js</code>	Escolhe prompt (integração, portable, defeito...)

Prompts (prompts/)

Prompt	Quando
<code>bdd-integracao.txt</code>	Worklist ↔ Portal, Leorad
<code>bdd-desktop-portable.txt</code>	Portable, laudário
<code>bdd-defeito.txt</code>	Resultado obtido
<code>bdd-refine.txt</code>	Refino OpenAI do rascunho
<code>bdd-evidencias.txt</code>	Com prints Dev
<code>bdd-objetivo.txt</code>	Anti-alucinação
<code>bdd-vocab.txt</code>	Vocabulário Mobilemed

Detecção automática: `BDD_PROMPT_AUTO=1`

Gravação no CRM

Situação	Comportamento
Campo Cenários QA vazio	Gera e grava
Campo já aprovado	Não altera
Gherkin inválido (# , ... , E cenário:)	Reescreve com BDD limpo
BITRIX_BDD_CLEAN_CRM_WRITE=1	Sem comentários # no campo

SPA 1294 → ufCrm100CenariosQa

SPA 1276 → ufCrm94CenariosQa

Comandos principais

```
cd ia-bdd-agent
npm run poll # Fila automática (~30s)
npm run bdd:item -- 1258 1294 # Um card (força CRM)
npm run bitrix:context # Estágios e SPAs
npm run bdd:prompts # Lista modos de prompt
npm run api # Servidor HTTP (Postman)
```

Saída: `output/bdd-{id}-*.feature` + consolidado `bdd-todas-*.feature`

Configuração essencial (`.env`)

```
BITRIX_WEBHOOK=https://...  
BITRIX_ENTITY_TYPE_IDS=1276,1294  
BITRIX_QA_STAGE_NAMES=Novo Teste,Teste de Q.A,...  
BITRIX_UF_BDD_FIELD=ufCrm100CenariosQa,ufCrm94CenariosQa  
  
BDD_USE_LLM=1  
OPENAI_API_KEY=sk-...  
BDD_LLM_REFINE=1  
BDD_PREFER_STRUCTURED=1  
BITRIX_BDD_CLEAN_CRM_WRITE=1
```

Reiniciar `npm run poll` após alterar `.env` .

Demo — o que mostrar

1. Terminal: `npm run poll` (horário **BRT**)
2. Card **novo** em Novo Teste → alerta na fila
3. Campo **Cenários QA** antes (vazio) e depois (Gherkin)
4. Arquivo `.feature` em `output/`
5. Card **já preenchido** → log *ignorado*, CRM intacto

Exemplo real: card **1258** — CNPJ indevido para Pessoa Física no Portal.

Exemplo — antes e depois (card 1258)

Antes (inválido):

- Comentários # Cenários QA , # Foco
- Quando desenvolvimento Web Portal... (truncado com ...)

Depois (executável):

Cenário: Pessoa Física no cadastro de unidades sem exigir CNPJ
Dado que o usuário acessa o ambiente Portal Web (Mobilemed)
E o usuário abre o cadastro de unidades
Quando o usuário seleciona o tipo "Pessoa Física" no formulário
Então o campo CNPJ não é exibido como obrigatório

Benefícios

Para QA	Para o time
Rascunho BDD na chegada do card	Menos retrabalho manual
Revisão no próprio Bitrix	Formato Gherkin padronizado
Não apaga cenários aprovados	Rastreio: logs + <code>.feature</code> + CRM
Então completos e verificáveis	Cobertura Dev mapeada nos cenários

Limitações (transparência)

- Qualidade depende dos **campos preenchidos** no chamado
- API Bitrix (503 / limite) → retry no próximo ciclo do poll
- LLM é **opcional**; estruturado funciona sem token
- Card **só com título** → cenário mínimo (melhor com NGF/Dev)
- **Não substitui** revisão humana — é rascunho para QA ajustar

Estrutura do repositório

```
qa-ai-agent/  
└─ ia-bdd-agent/  
    ├── poll.js, index.js, api-server.js  
    ├── docs/APRESENTACAO-MARP.md    ← esta apresentação  
    ├── prompts/                     ← templates LLM  
    ├── output/                       ← .feature gerados  
    ├── scripts/                     ← bdd:item, calibrate  
    └── src/  
        ├── agents/                  ← bdd, parser  
        ├── services/                ← Bitrix, CRM, IA  
        ├── orchestrator/            ← ciclo BDD  
        └── utils/                   ← Gherkin, rigor, planner
```

GitHub: github.com/victor-martinss/ia-bdd-agent

Encerramento

ia-bdd-agent em uma frase

Automatiza a primeira versão dos cenários QA a partir do que já está no Bitrix — na hora em que o card entra na fila de teste.

Próximos passos

- Webhook Bitrix ao mover card → Novo Teste
- Integração com runner Cucumber / CI
- Filtros por esteira (`BITRIX_QA_CATEGORY_NAMES`)

Dúvidas?